DATA 260 Homework 9

Pragya Apurva | Rental housing listings | November 7, 2026

Repository: https://github.com/ishuapurva1996/data260-6102

Section 0	Value	Section 0	Value
SID4	6102	PORT_BASE	8702
PREFIX	s6102	SEED	6102
VERIFY_SEED	266102	DOMAIN_ID	6

Runtime and submission metadata

Apple M4 MacBook Air, 10 CPU cores, 24 GiB RAM; macOS 15.7.4 arm64; Python 3.12.13. Local Ollama 0.33.0 serves **qwen2.5:7b (Q4_K_M)**. Generation uses temperature 0, seed 6102, context 4096, and at most 256 output tokens. Part 4 requests JSON.

The integrated application uses **port 8702, threshold 0.85, and version enforcement enabled**. MySQL 8.4, Kafka 3.9.2, and Redis Search 8.2.3 run in isolated services. Nomic nomic-embed-text:v1.5 supplies 768-dimensional float32 vectors with the prefix "clustering: "; Redis uses FLAT cosine search. Full model digests are in run metadata.

Integration code/source: 958af31b31a54fb6dc928f54a6c41d73e3f82d46
Branch: codex/hw9-integration. Final local submission tag: **hw9**. The tag is created after this PDF is final; the upload package records its exact target in TAG_TARGET.json.

Evidence	Measured source / retained branch tip
Part 1	4738560fb54e / A: bd82926172d8
Part 2	4bc060d11adb / B: 6938c7111084
Part 3 sweep and stale demo	52c8ef470b12 / C: f7e5af838ede
Part 4 development and test	7767adc8fdc4 / D: c00f72e9dc1e

Integration preserved B in C's ancestry and the pre-sweep YAML commit 3511ae67588f61eeff4199aa8f4db294aca04c57. Measured cache, embedding, generation and Part 4 code remains unchanged. The report retains those original measurements; new live checks verify the combined system.

The repository link is accessible. Versioned file links in this PDF refer to the local hw9 tag and become available online after authorized publication. Submit Apurva_HW9.pdf with the same bytes as the tagged report. Remote publication and Canvas upload await separate instructions; check Canvas for the due date and any naming override. Setup and verification guide. Rechecked November 7: Sbnikitha and supriyaselvanganesan appear as direct collaborators.

Part 1 - Redis rental view counter

The increment_rental_views method counts one successful authenticated detail GET as one view. Collection reads, missing listings and rejected authentication do not. The key is s6102:views:rental:<id>. A missing key starts from the latest MySQL checkpoint using SET NX and an atomic transaction; warm keys use a Lua INCR operation.

Code and observed checkpoints

Abridged excerpts from rental_views.py; the SQL update only advances a lower saved count.

```
key = f"s6102:views:rental:{rental_id}"
transaction.set(key, saved_count, nx=True)
transaction.execute_command("INCR", key)
# After the atomic increment:
if count % 10 == 0:
    db.commit()
```



Figure 1. Saved-record evidence view of A's live HTTP transcript and FastAPI checkpoint log. Pragya Apurva is visible; the complete 1-25 sequence is retained in the raw log.

At reads 9, 10, 20 and 25, MySQL held 0, 10, 20 and 20; Redis ended at 25. A second listing stayed independent. Concurrent cold reads resumed at 21-28, and an older SQL snapshot resumed from checkpoint 30 at 31. The integrated verifier reproduced these controls.

Trade-off: losing Redis after read 25 loses the five unsynced views after checkpoint 20. Redis errors surface as request errors. Counter-only SQL writes preserve the rental-fact cache version. Evidence: counter JSON and integrated live receipt.

Part 2 - Semantic response cache

The authenticated POST /api/kafka/questions route embeds the incoming question, searches eligible Redis answers, and compares cosine similarity with 0.85. A hit returns the stored final answer before publishing to Kafka. A miss reads current rental facts, runs Planner, Writer and Reviewer through the existing adapter, and caches only an approved final answer.

Code and cache-bypass evidence

Abridged excerpts from SemanticCache.closest and the Q&A handler. The real Redis KNN query filters by dataset, user and current data version before ranking.

```
filters += f" @data_version:{{{version}}}"
# Redis search: filtered KNN 1, COSINE distance
if is_cached:
    answer = candidate["answer"]
else:
    result = await send_question(...)
```

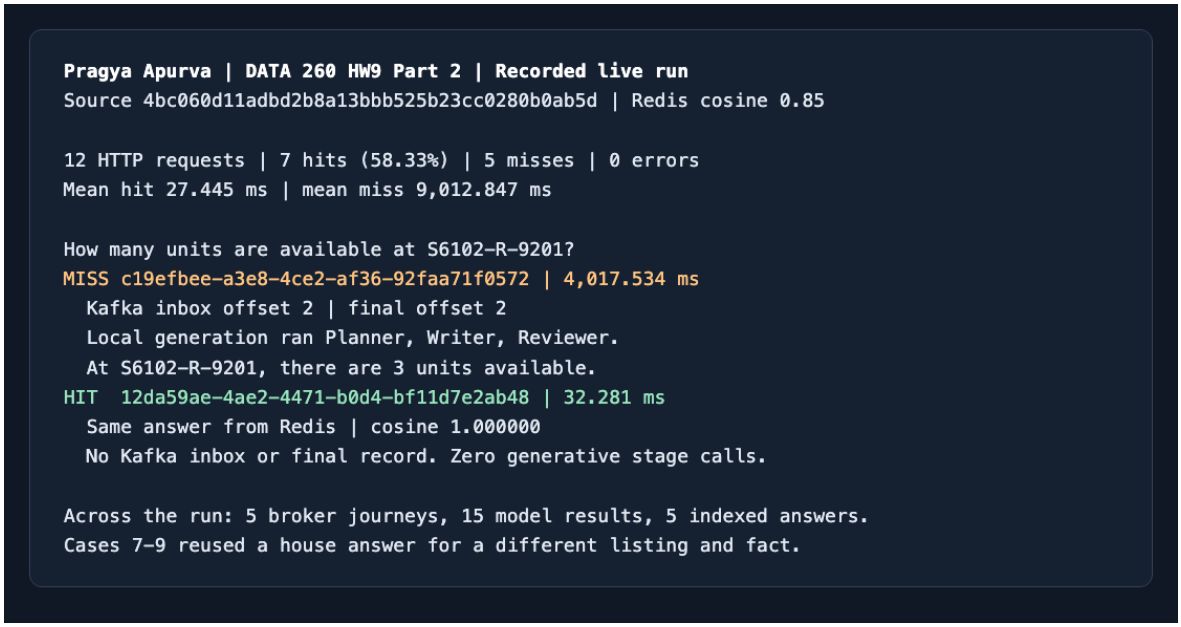


Figure 2. Saved-record evidence view of B's recorded requests and joined broker/model logs. Every request receives a fresh ID. A hit still calls the embedder; it skips the three generative stages.

Requests	Hits / rate	Misses	Mean hit (ms)	Mean miss (ms)
12	7 / 58.33%	5	27.445	9,012.847

All 12 requests succeeded, including exact duplicates, paraphrases and new questions. All five misses had one inbox and final record plus three real model results. All seven hits had no such records or generation calls. Responses include answer, is_cached, similarity, latency_ms and request_id; correlated logs retain stage times.

Correctness limitation: cases 7-9 incorrectly reused a property-type answer for another listing's title. These three wrong cached answers remain in the hit rate. The new integrated check separately confirmed unchanged broker offsets across an exact hit. Evidence: requests and bypass verification.

Part 3 - Threshold economics

The frozen YAML contains 40 labeled inputs: 10 exact duplicates, 10 paraphrases, 10 superficially similar questions with different answers, and 10 unrelated questions. Ten verified live seed answers were restored into five fresh namespaces. All 200 HTTP requests succeeded. Order, fixtures, account, version, model and seed were fixed; normal cache writes remained enabled within each pass.

Frozen SHA-256:
4c180dd5a8b5dbd007797225889d1cb7e0921339d0670683b225674ee76f0c26

Cutoff	Hit rate	False-hit rate	Mean HTTP ms	Cost (s-eq)
0.75	95.0%	52.5%	663.40	6310
0.80	85.0%	42.5%	2,078.91	5130
0.85	75.0%	32.5%	3,832.83	3950
0.90	72.5%	37.5%	4,131.69	4555
0.95	72.5%	37.5%	4,254.44	4555

Hit rate = cached answers / 40. False-hit rate = factually incorrect cached answers / 40, rather than only the negative labels or only cache hits. At **0.85, 30/40 were hits and 13/40 were incorrect cached answers** (32.5%). Of the 20 should-miss cases, 50% were accepted at that cutoff. No wrong live answers were observed in this sweep.

Fixed cost model and choice

A cache miss costs **5 seconds-equivalent**, a nominal local generation budget. An incorrect cached answer costs **300 seconds-equivalent**, an assumed five-minute support correction. Both are fixed illustrative weights declared in the YAML, rather than measured support time or financial charges. The 5-second miss weight understates observed miss latency.

```
total_cost = misses * 5 + incorrect_cached_answers * 300
```

Choose **0.85**, the lowest cost among the five tested settings: 10 x 5 + 13 x 300 = **3,950 seconds-equivalent**. The predeclared tie-break chooses the higher cutoff. A separate smoke run produced a fresh miss and then a hit. This choice remains error-prone and is not a claim of production readiness.

Two scoring corrections

The automatic keyword screen wrongly accepted (1) a deposit answer that mentioned wear and tear without defining it, and (2) a lease answer that mentioned responsibilities without explaining a co-signer. The full-answer audit corrected those two correctness decisions. The original automatic records, all expected facts and all should-hit labels remain unchanged.

At 0.90 and 0.95, P02 misses at similarity 0.876015127 and adds a new answer. P04 then matches that different listing at 0.999324322. The changing cache contents within a pass explain why a higher cutoff can still yield more wrong hits.

Final table sources: audited summary and audited requests. Audit: two corrections and all 200 decisions. Inputs: cache_cases.yaml and fixture facts.

Part 3 - False hit and stale-answer protection

One observed false hit in full

Cached: How many units are available at S6102-R-9102?
Incoming: How many units are available at S6102-R-9106?
Cosine similarity: 0.999333381652832; cutoff 0.75; frozen case N01.

Returned cached answer: "The rental housing draft is clear and accurate. Here is the final answer: At S6102-R-9102, there are 4 units available."
Correct answer: S6102-R-9106 has 1 available unit.

Only the listing code changed in an otherwise identical question template. The embedding treated them as nearly identical, although the records had different facts. Complete false-hit trace.

Stale 3 after the persisted value became 1

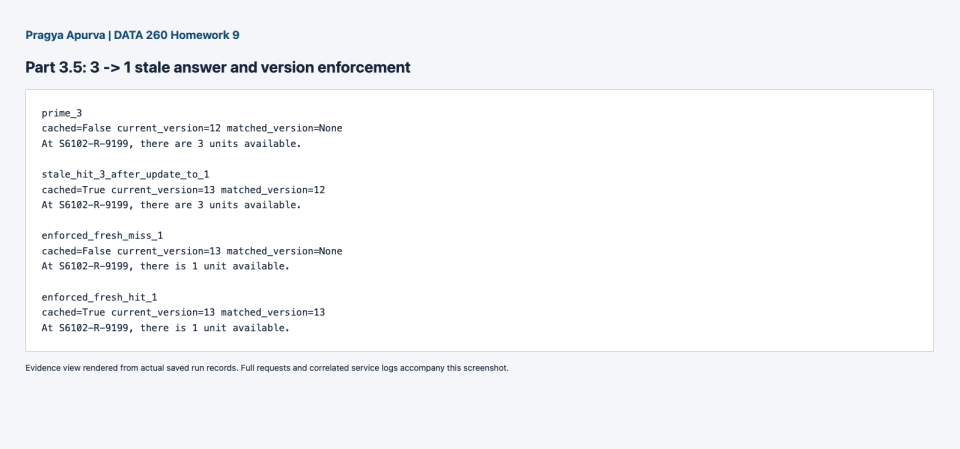


Figure 3. Saved-record evidence view of the four stale-answer requests; name, cached flags, versions and returned counts are visible.

The isolated stale-demo namespace alone bypassed version filtering. An authorized SQL PUT changed S6102-R-9199 from 3 to 1. Restarting the same namespace with enforcement produced a miss at 1, then a hit at 1. Full four-step trace and persisted before/after facts.

Transactional invalidation and integration

Abridged excerpts from rental write handlers and rental_facts.py:

```
advance_rental_version(db)
db.commit()
# Version increment is in the same SQL transaction.
# Cache lookup filters to the current version.
```

Creates, updates and deletes advance one dataset-wide version in their write transaction. Duplicate-code errors roll back both facts and version. A response generated while the version changes is not stored (skipped_version_changed). Saved B evidence checks that race and that a current eligible vector wins over a stale exact match.

The integrated live test confirms that counter checkpoint writes leave the fact version unchanged and keep the answer eligible, while a relevant 3-to-1 listing update advances it and forces a fresh answer. **Trade-off:** one listing change invalidates older answers for every listing, lowering reuse. Versioning does not resolve cross-listing false hits. Evidence: race/filter checks and integrated checks.

Part 4 - Two agents, repair and lesson reuse

The Answering Agent sends the question, source fields, applicable lessons, and any prior feedback to local Qwen through the shared adapter. The Evaluator Agent is a **deterministic reference checker**: it compares requested JSON fields and types with canonical facts and returns PASS or REVISE with a reason and corrective feedback. It is a separate agent role, without an additional evaluator-model call.

Short agent excerpts and complete revision evidence

Abridged excerpts from src/hw09_agents.py; omitted branches handle malformed output and exact field/type checks.

```
payload = {"input": question, "listing_source": source}
payload["evaluator_feedback"] = prior.grade.feedback
result = self.model.complete(...)

missing = expected.keys() - claims.keys()
extras = claims.keys() - expected.keys()
if not (missing or extras or wrong):
    return Evaluation("PASS", ...)
```

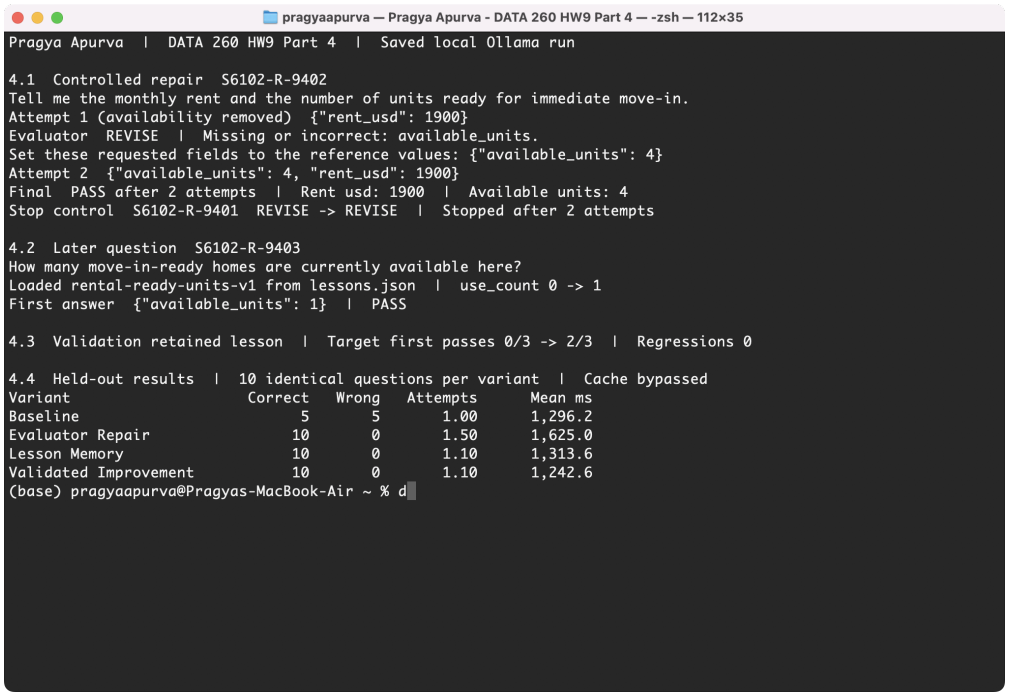


Figure 4. Terminal capture displaying D's saved local-model records. Pragya Apurva is visible. It contains the full question, intentionally incomplete draft, feedback, revised answer, final PASS, attempt count and final response; it also shows the stop and later lesson-reuse controls.

For S6102-R-9402, the controlled first draft omitted availability. Feedback requested 4 units; attempt 2 returned rent 1900 and availability 4, then PASS. For S6102-R-9401, both attempts were deliberately stripped; the final result remained REVISE after two attempts. Controlled omissions were confined to these demonstrations.

A repeated training error created rental-ready-units-v1: use inventory_export.bucket_b for immediate availability, including zero. The later S6102-R-9403 question reloaded it, passed on its first answer, and raised use_count from 0 to 1. Full revision, stop and reuse traces; lesson with all five required fields.

Part 4 - Held-out results and validation

Variant	Correct	Incorrect	Avg. attempts	Avg. latency ms
Baseline	5	5	1.00	1,296.2
Evaluator Repair	10	0	1.50	1,625.0
Lesson Memory	10	0	1.10	1,313.6
Validated Improvement	10	0	1.10	1,242.6

Each row independently used the same 10 held-out inputs, model and fact rubric, with the semantic cache bypassed. Baseline generated once, without feedback or lessons. Repair allowed two attempts. Memory used the training lesson; Validated Improvement used only the accepted lesson. Testing froze lesson snapshots and disabled lesson edits. The two lesson variants used the same accepted lesson; their latency difference comes from separate runs.

Latency includes generation, applicable lesson loading, in-loop evaluation and repair, and final formatting. It excludes warm-up and offline grading. Offline grading did not send feedback to baseline. Correct plus incorrect equals 10 in every row. The dedicated timing audit observed 48 chat calls, matching 47 answer attempts plus one warm-up.

Validation and preserved data

Paired validation with repair enabled retained the lesson: target first-answer passes improved from 0/3 to 2/3; final passes stayed 6/6; unrelated questions gained no unsupported claims. The zero-availability case still needed feedback to correct 3 to 0. That limitation remains in the saved pairs.

The reference supplies ground truth. Input-only training, validation, and test cases are disjoint. The runner opened held-out inputs after freezing prompts, rubric, code and lessons. Integration reruns development controls only.

Held-out SHA-256:
49c667b18a848fdb3756805bcaf6d27a658c307f6b38a2c943ed270a96f615b3

Limitations: small synthetic structured cases, one timed pass, reference-assisted repair that supplies correct values, and dependence on complete reference facts. Local latency has no confidence interval. The active suites passed 1,584 checks in application and dedicated MCP environments; 25 optional older MySQL checks were skipped. Verification receipt.

AI-use disclosure

1. Assistance and work: Codex helped write and integrate code, fixtures, runners, the verifier and report. Actual local outputs and frozen inputs support the results.
2. Unsuitable result: two keyword scores accepted incomplete definitions, and an initial verifier PUT omitted required fields.
3. Checks: fact/rubric audits, independent recomputation and regrading, real HTTP/Redis/MySQL/Kafka checks, tests, source review and visual PDF inspection, executed with AI assistance.
4. Changes: corrected only audited correctness decisions, fixed the verifier requests and fixture lookup, and added setup/verification. Labels, measured code and held-out inputs stayed intact. Full disclosure.